From Observability to Verifiability: An Execution Evidence Architecture for Agentic Software Systems

Anonymous Draft

March 2026

Abstract

Agentic software systems are increasingly expected to invoke tools, consume budgets, and produce externally consequential outputs, yet most current runtimes still expose execution primarily through logs and traces. Those artifacts are useful for observability, but they do not by themselves provide a bounded, reviewable, and independently checkable account of what was intended, what was authorized, what was executed, and what evidence was exported. This paper frames that mismatch as an execution evidence problem rather than a logging problem.

We present an execution evidence architecture for agentic software systems and instantiate it as a reproducible reference path in the `verifiable-agent-demo` repository. The architecture links five stages of runtime semantics: persona projection, intent/action/result objects, governance checkpointing, execution-integrity validation, and bounded audit receipts. The implementation is evaluated through a deterministic 16-task suite, a unified five-file export contract, a third-party bundle review script, a CrewAI-like external baseline, four architectural ablations, and a blinded human-review kit.

Across the checked-in evaluation artifacts, the full evidence-chain configuration improves average explicitness from 1.0 to 5.0, replayability from 3.0 to 5.0, tamper sensitivity from 0.0 to 4.81, audit boundedness from 2.0 to 5.0, and integration surface from 1.0 to 5.0 relative to the local trace-oriented baseline. The external baseline reaches replayability 4.0, but leaves the other evidence dimensions at trace-like levels. Ablation results show targeted degradation when intent capture, governance persistence, integrity validation, or receipt export is removed. A lightweight blinded review kit is included as a reproducible study scaffold; the synthetic smoke-test bundle sheet indicates that execution verification is the most error-prone review question. These results suggest that execution evidence should be treated as a distinct systems layer between runtime observability and audit-oriented verification.

Keywords. agentic software systems, execution evidence, audit receipts, verifiability, runtime governance

1 Introduction

Agentic software systems are increasingly used to execute software tasks, invoke tools, consume budgets, and produce outputs that matter beyond the runtime boundary. As these systems move from laboratory demos to operational workflows, the engineering question is no longer only whether an agent can complete a task. It is also whether the resulting run can be independently reviewed, replayed, and trusted by a third party.

Current practice usually answers this need with observability artifacts such as logs, callback streams, trace records, and framework-specific execution histories. These artifacts are valuable for debugging and operational visibility, but they are not the same as execution evidence. A trace can show that something happened without making the requested action explicit, without

preserving a governance decision, without exposing whether execution integrity was checked, and without exporting a bounded receipt that a reviewer can inspect without reconstructing the full runtime context.

This distinction matters because inspectability is weaker than verifiability. An operator may be able to browse a log and infer what likely happened, but an independent reviewer still lacks a stable artifact contract for answering four basic questions: was intent captured, was policy checked, was execution verified, and was a receipt exported? When those questions are left implicit, a run can be observable without being verifiable.

We refer to this mismatch as the *execution evidence gap*. The gap appears whenever a run claims that a task was requested, authorized, executed, and completed, yet the exported artifacts are insufficient to confirm those claims in a structured and reproducible way. In agent stacks this gap is easy to miss because observability tooling is often rich, while audit-oriented evidence surfaces remain ad hoc.

This paper presents an execution evidence architecture for agentic software systems. The proposal is not a new orchestration framework. Instead, it is a composable systems layer that links five stages of runtime semantics: persona projection, intent/action/result objects, governance checkpoints, execution-integrity validation, and bounded audit receipts. The purpose of the architecture is to make agent runs exportable as reviewable evidence bundles.

We instantiate the architecture as a reference path in the public `verifiable-agent-demo` repository. The checked-in implementation includes a deterministic 16-task suite, a unified five-file export contract, a third-party review script, a trace-oriented baseline, a CrewAI-like external baseline, four ablation modes, and a blinded human-review kit. The resulting harness allows the paper to evaluate execution evidence as an implemented systems concern rather than as a purely conceptual abstraction.

The evaluation compares seven conditions: a local trace-oriented baseline, an external framework-style baseline, four evidence-chain ablations, and the full evidence-chain configuration. Across these settings we measure five dimensions: explicitness, replayability, tamper sensitivity, audit boundedness, and integration surface. The checked-in summaries indicate that the full evidence chain improves every metric over the trace-oriented baseline, while the external baseline primarily improves replayability without recovering the missing evidence semantics. The ablation results further show targeted degradation when individual stages are removed.

The contributions of this paper are threefold:

- It formulates execution evidence as a software systems problem that is distinct from runtime observability.
- It defines a five-stage execution evidence architecture that links pre-execution semantics, governance, integrity validation, and bounded receipts.
- It implements and evaluates a reproducible reference path showing that evidence-oriented exports outperform trace-only baselines on explicitness, replayability, tamper sensitivity, and audit readiness.

The remainder of the paper is organized as follows. Section 2 introduces the method and reference architecture. Section 3 describes the evaluation setup. Section 4 reports the results across baselines, ablations, and review workflows. Section 6 discusses limitations, boundary conditions, and implications for software engineering practice.

2 Method

2.1 Problem framing

The goal of this work is to define and evaluate a minimal execution evidence architecture for agentic software systems. The target problem is narrower than model correctness and broader than debugging. We do not attempt to prove that an agent's output is semantically correct, and we do not claim formal verification of arbitrary tool behavior. Instead, we ask a systems question: how should runtime artifacts be organized so that an independent reviewer can determine whether intent was captured, policy was checked, execution was verified, and evidence was exported?

This framing deliberately separates execution evidence from raw observability. Logs and traces remain useful, but they do not by themselves guarantee a stable review surface. The proposed method therefore centers on exported artifact contracts rather than hidden runtime state.

2.2 Execution evidence architecture

We define the architecture as a five-stage evidence chain:

1. persona projection,
2. intent/action/result objects,
3. governance checkpoint,
4. execution-integrity validation,
5. bounded audit receipt.

Persona projection. Persona projection provides the runtime-facing identity context of the acting system. In the current repository it functions as an entry-stage semantic anchor rather than as a separate evaluation target. The architecture needs a notion of who or what is acting, even when the downstream evidence metrics focus on later stages.

Intent/action/result objects. Intent, action, and result objects convert a run from an implicit event sequence into structured semantics. The intent object captures the declared goal and expected artifact surface before execution. The action object records the operational step together with authorization-relevant context. The result object captures terminal state, including successful completion, policy blocking, and tamper-detected outcomes.

Governance checkpoint. The governance layer records whether a run was checked against policy. In the current harness this includes budget-aware and approval-aware blocking cases. The key design choice is that a blocked run is still evidence-bearing: policy denial is preserved as an explicit outcome instead of disappearing as missing output.

Execution-integrity validation. The integrity layer records whether the execution path was checked for replay-oriented consistency or tamper-sensitive mismatch. This layer is intentionally bounded. It does not provide a universal proof system; it provides enough structure to distinguish completed runs, non-verified runs, and tamper-detected runs within the artifact bundle.

Bounded audit receipt. The bounded receipt is the review-oriented export surface. Unlike a raw trace, it preserves only the fields needed to answer downstream audit questions. The receipt therefore compresses the run into a stable, third-party-facing representation.

Together these stages convert an agent run from framework-specific observability into a portable evidence bundle.

2.3 Reference path

The architecture is instantiated in the `verifiable-agent-demo` repository as a composable reference path. The implementation reuses concepts and naming from adjacent repositories, but keeps the evaluation logic centered in a deterministic local harness. This choice reduces hidden dependencies and makes every condition reproducible from checked-in task specifications and scripts.

For every task run, the harness exports exactly five JSON files:

- `intent.json`,
- `action.json`,
- `result.json`,
- `trace.json`,
- `receipt.json`.

These files are written under the stable layout `artifacts/runs/<task_id>/<mode>/`. The contract is shared across all conditions, including the baselines and ablations. This makes it possible to compare evidence shape directly without writing a framework-specific parser for each runtime condition.

2.4 Task suite

The evaluation harness includes 16 machine-readable tasks grouped into five categories:

- four read-only query tasks,
- four tool-call tasks,
- three budget-constrained tasks,
- three approval-required tasks,
- two tamper scenarios.

Each task specification records a task identifier, category, title, goal, input payload, expected artifacts, budget policy, approval policy, tamper case indicator, and notes. The mix of task classes is important because an evidence architecture should represent both positive and negative execution paths. In particular, the budget, approval, and tamper tasks test whether blocked and invalid runs appear as explicit outcomes rather than as absent output.

2.5 Experimental conditions

We evaluate seven conditions:

Baseline. The baseline is a local trace-oriented path with minimal evidence semantics. It approximates trace-only export without explicit intent capture, governance persistence, integrity validation, or bounded receipts.

External baseline. The external baseline is a lightweight CrewAI-like native logging baseline implemented as a deterministic local adapter. It preserves a framework-style evidence surface based on agent initialization, task dispatch, callbacks, and task completion, but it does not claim live end-to-end CrewAI governance or receipt support.

Evidence chain. The evidence-chain condition is the full proposed architecture, including explicit intent capture, policy persistence, integrity validation, and bounded receipt export.

Ablations. The four ablations each remove one capability from the evidence chain: `no_intent`, `no_governance`, `no_integrity`, and `no_receipt`. These conditions isolate whether each stage contributes a distinct review function instead of merely adding redundant metadata.

2.6 Review logic and metrics

A third-party review script consumes a run directory and answers four yes/no questions:

1. was intent captured,
2. was policy checked,
3. was execution verified,
4. was a receipt exported.

The script emits machine-readable verdicts, an overall status, missing-field explanations, and a short summary. This review step operationalizes the minimum independent inspection workflow that the architecture is intended to support.

We compare conditions using five deterministic, rule-based metrics derived from actual generated artifacts:

- **explicitness**: whether intent, policy, and outcome are directly inspectable;
- **replayability**: whether the run preserves enough structure for replay-oriented checking;
- **tamper sensitivity**: whether integrity-relevant violations appear as explicit outcomes;
- **audit boundedness**: whether the review surface is compact and purpose-oriented;
- **integration surface**: how completely the harness exposes the intended evidence stages.

Some parts of these metrics are directly measurable from the exported fields, while others are scored by transparent 0–5 rules documented in the repository. The method does not report semantic-quality scores or invented empirical values.

2.7 Human review scaffold

In addition to automated review, the repository includes a blinded human-review kit. The kit contains reviewer instructions, blinded bundle manifests, blank review sheets, timing guidance, and a summarizer for completed sheets. The current checked-in results are synthetic smoke-test outputs rather than a completed human-subject study. The role of the kit in this paper is therefore methodological: it demonstrates that the artifact bundle is structured for human inspection and that the workflow is reproducible without requiring a web platform.

3 Evaluation Setup

3.1 Study goal

The evaluation asks whether execution evidence should be treated as a distinct systems layer rather than as an extension of runtime logging. Concretely, we test whether the full evidence chain yields artifact bundles that are more explicit, more replayable, more tamper-sensitive, and more audit-ready than baseline observability surfaces.

3.2 Repository-grounded setup

All measurements in this draft come from the checked-in repository state. The harness is implemented in `verifiable-agent-demo` and exercised through the top-level make targets documented in the repository `README`. The evaluation sources are:

- the 16-task suite under `evaluation/tasks/`,
- the five-file export contract under `artifacts/runs/<task_id>/<mode>/`,
- comparison summaries under `artifacts/metrics/`,
- paper-support documentation under `docs/paper_support/`,
- the human-review kit under `evaluation/human_review/`.

The repository currently supports the following relevant workflows:

- `make paper-eval`,
- `make eval-external-baseline`,
- `make eval-ablation`,
- `make human-review-kit`,
- `make top-journal-pack`.

3.3 Task distribution

The task suite is intentionally heterogeneous. Read-only and tool-call tasks cover ordinary successful runs. Budget-constrained and approval-required tasks exercise policy gating. Tamper scenarios exercise integrity-sensitive outcomes. Because the same 16 task specifications are used in every condition, differences in score are attributable to the exported evidence surface rather than to different workloads.

3.4 Conditions compared

The core automated comparisons are:

- **baseline** versus **evidence_chain**,
- **external_baseline** versus **evidence_chain**,
- a seven-mode comparison across baseline, external baseline, four ablations, and the evidence chain.

The main comparison asks whether the full architecture improves over a local trace-oriented export. The external baseline comparison asks whether a framework-style default logging surface closes the same gap. The ablation study asks whether removing any one stage causes interpretable degradation.

3.5 Outcome interpretation

The review script distinguishes among several outcome classes. A run may be marked *pass* when all four evidence questions are satisfied, *policy_blocked* when a budget or approval checkpoint blocks the task but still leaves reviewable evidence, *tamper_detected* when the integrity layer detects mismatch, or *partial* when one or more evidence-chain capabilities are missing.

This outcome model is important because it treats blocked and tampered runs as first-class results. In the full evidence chain, the repository summaries show 11 completed and review-passing runs, 3 policy-blocked runs, and 2 tamper-detected runs across the 16-task suite.

3.6 Human-review procedure

The blinded human-review kit packages eight sample bundles, reviewer instructions, a blank CSV review sheet, and an answer key used only by the summarizer. Reviewers inspect five JSON files per bundle and answer the same four questions used by the automated review script, while recording confidence and time spent. The repository includes a synthetic example sheet and synthetic summary outputs so that the workflow can be smoke-tested end to end.

4 Results

4.1 Baseline versus full evidence chain

Table 1 reports the main baseline comparison. The full evidence chain improves every measured dimension over the trace-oriented baseline. Intent capture, policy checking, and receipt export all move from zero observed positive reviews in the baseline to 16 of 16 tasks in the evidence chain. Execution verification rises from 0 of 16 to 11 of 16 because the remaining five tasks are intentionally represented as three policy-blocked outcomes and two tamper-detected outcomes rather than as successful verified completions.

The metric changes are large. Average explicitness rises from 1.0 to 5.0, replayability from 3.0 to 5.0, tamper sensitivity from 0.0 to 4.81, audit boundedness from 2.0 to 5.0, and integration surface from 1.0 to 5.0. This pattern supports the central claim of the paper: the full evidence chain is not merely a richer trace, but a different review surface.

Table 1: Trace-oriented baseline versus full evidence chain. Counts report positive bundle-review verdicts across the 16-task suite; scores are average rule-based metrics on a 0–5 scale.

Mode	Intent	Policy	Exec. verified	Receipt	Explicitness	Replayability	Tamper	Audit	Integration
Baseline	0/16	0/16	0/16	0/16	1.0	3.0	0.0	2.0	1.0
Evidence chain	16/16	16/16	11/16	16/16	5.0	5.0	4.81	5.0	5.0

Table 2: CrewAI-like external baseline versus the full evidence chain. The external baseline preserves a richer default execution surface than the local trace baseline, but does not add explicit evidence-chain semantics.

Mode	Intent	Policy	Exec. verified	Receipt	Explicitness	Replayability	Tamper	Audit	Integration
External baseline	0/16	0/16	0/16	0/16	1.0	4.0	0.0	2.0	1.0
Evidence chain	16/16	16/16	11/16	16/16	5.0	5.0	4.81	5.0	5.0

4.2 External baseline comparison

Table 2 compares the CrewAI-like external baseline with the full evidence chain. The external baseline improves replayability relative to the local baseline, reaching an average score of 4.0, because the framework-style adapter preserves a somewhat richer default execution surface. However, it leaves explicit intent capture, policy persistence, integrity verification, and receipt export absent. As a result, explicitness, tamper sensitivity, audit boundedness, and integration surface remain at 1.0, 0.0, 2.0, and 1.0 respectively.

This result matters because it reduces the risk that the main comparison only beats a straw baseline. A more realistic framework-native logging surface still does not recover the evidence-chain properties by default.

4.3 Ablation results

Table 3 summarizes the seven-mode comparison. The ablations show targeted degradation rather than uniform collapse.

Removing intent capture lowers explicitness from 5.0 to 3.0 and integration surface from 5.0 to 4.0, while leaving policy, integrity, and receipt behaviors mostly intact. Removing governance lowers explicitness to 2.0, eliminates all positive policy-checked reviews, and changes the task suite behavior by allowing formerly blocked tasks to complete, while tamper sensitivity remains high because integrity checks are still present. Removing integrity leaves explicitness and audit boundedness at evidence-chain levels, but drives tamper sensitivity and execution verification to 0.0 and 0 of 16. Removing receipt export preserves most other capabilities but drops audit boundedness from 5.0 to 2.0 and receipt-exported reviews from 16 of 16 to 0 of 16.

Taken together, the ablations indicate that each stage contributes a distinct review function. The evidence chain therefore behaves as a layered architecture rather than as a single monolithic switch.

4.4 Human-review kit observations

The repository's human-review result is currently a synthetic smoke test rather than a real reviewer study, so it should be interpreted as a workflow check, not as a behavioral claim about human subjects. The synthetic summary covers eight blinded bundle reviews, split evenly between baseline and evidence-chain conditions. In that synthetic example, both conditions reach

Table 3: Seven-mode comparison across the baseline, external baseline, four ablations, and the full evidence chain. Scores are average 0–5 values; counts report positive review verdicts across 16 tasks.

Mode	Explicitness	Replayability	Tamper	Audit	Integration	Intent	Policy	Exec. verified	Receipt
Baseline	1.0	3.0	0.0	2.0	1.0	0/16	0/16	0/16	0/16
External baseline	1.0	4.0	0.0	2.0	1.0	0/16	0/16	0/16	0/16
No intent	3.0	4.0	4.81	4.0	4.0	0/16	16/16	11/16	16/16
No governance	2.0	5.0	5.0	4.0	4.0	16/16	0/16	14/16	16/16
No integrity	5.0	5.0	0.0	5.0	4.0	16/16	16/16	0/16	16/16
No receipt	4.0	4.0	4.81	2.0	4.0	16/16	16/16	11/16	0/16
Evidence chain	5.0	5.0	4.81	5.0	5.0	16/16	16/16	11/16	16/16

question accuracy 0.9375, mean confidence 3.5, and the only recurring error class is execution verification. Mean review time is 117.5 seconds for baseline bundles and 161.5 seconds for evidence-chain bundles.

These numbers are not offered as empirical user-study findings. They show only that the blinded review workflow, answer key, and summarizer operate end to end, and that execution verification is a plausible stress point for future human evaluation.

5 Related Work

This work sits at the intersection of three familiar areas in software engineering systems: runtime observability, audit-oriented provenance, and agent or workflow orchestration. Existing observability practice emphasizes logs, traces, callback streams, and runtime telemetry as tools for debugging and operations. Audit-oriented systems, by contrast, emphasize bounded evidence, reproducible records, and post hoc review. Agent runtimes add a further layer of complexity because action planning, tool use, policy gating, and execution outcomes may be split across several framework surfaces.

The contribution of this paper is not to replace those strands, but to connect them. The execution evidence architecture argues that there is a missing layer between runtime visibility and audit review: a stable artifact contract that makes intent, governance, integrity, and receipts legible as first-class exported objects.

The current manuscript intentionally avoids fabricated citations. Before submission, this section should be grounded with verified references covering representative work in:

- runtime logging, tracing, and callback-based observability;
- provenance, accountable workflows, and audit-receipt style exports;
- agent and workflow runtimes that expose task histories but not full evidence bundles.

With those references in place, the paper's positioning claim is straightforward: observability surfaces and audit surfaces overlap, but they are not interchangeable, and agentic systems need an explicit bridge between them.

6 Discussion and Threats to Validity

6.1 What the results support

The repository results support a bounded systems claim: execution evidence can be modeled as a distinct layer above runtime observability and below downstream audit interpretation. In the implemented harness, this layer improves pre-execution explicitness, preserves governance outcomes as evidence-bearing states, detects tamper scenarios as explicit review results, and exports bounded receipts that a third party can inspect without reading a full trace transcript.

6.2 What the results do not support

The results do not show semantic correctness of model outputs. They do not provide formal verification of arbitrary agent behavior. They do not establish that the current external baseline is a complete representation of any production framework, and they do not yet provide a real human-subject study. The paper should therefore be read as a reference-architecture and reference-implementation study, not as a claim of universal runtime coverage.

6.3 Threats to validity

Construct validity. The metrics are evidence-shape metrics rather than semantic task-quality metrics. This is deliberate, because the paper studies reviewability and verifiability surfaces. Still, the conclusions should be limited to those properties.

Internal validity. The harness is deterministic and local. That improves reproducibility, but it also means that the system does not exercise the nondeterminism, service failures, and integration drift of production agent stacks.

External validity. The external baseline is a lightweight CrewAI-like adapter, not a deep live integration. This makes the comparison reproducible and honest, but it limits how far the result can be generalized to all framework-native logging surfaces.

Human-review validity. The current human-review workflow is a blinded kit plus a synthetic smoke test. It demonstrates process reproducibility, but it does not yet support claims about reviewer performance, statistical significance, or organizational adoption.

6.4 Implications for software engineering

Despite these limits, the reference path is useful. It suggests that software engineering for agentic systems should distinguish between *runtime telemetry*, which helps operators watch a system, and *execution evidence*, which helps reviewers verify what the system claimed to do. That distinction matters in settings where tool use, budgets, approvals, and external outputs need a compact evidence trail rather than an unbounded stream of internal events.

7 Conclusion

This paper presented an execution evidence architecture for agentic software systems and instantiated it as a reproducible reference path in the `verifiable-agent-demo` repository. The central

claim is that observability alone is not enough: agent runs need structured evidence bundles that make intent, authorization, execution verification, and exported receipts independently reviewable.

The checked-in evaluation harness supports that claim with a deterministic 16-task suite, a stable five-file export contract, a third-party review script, a CrewAI-like external baseline, four ablation modes, and a blinded human-review kit. Across the current summaries, the full evidence chain dominates the trace-oriented baseline on all measured dimensions and shows interpretable degradation under targeted ablations.

Future work should validate the architecture against live framework integrations, extend the integrity layer beyond the current bounded checks, add verified related-work citations, and run a real blinded reviewer study. Even in its current form, the reference implementation shows that execution evidence can be engineered as a first-class systems layer rather than treated as a side effect of logging.

A Reproduction Notes

The manuscript is grounded in the repository state rather than in a separate private experiment log. The relevant source material includes:

- `docs/paper_support/task-suite.md`,
- `docs/paper_support/export-format.md`,
- `docs/paper_support/review-workflow.md`,
- `docs/paper_support/comparison-workflow.md`,
- `docs/paper_support/external-baseline.md`,
- `docs/paper_support/ablation-study.md`,
- `docs/paper_support/human-review-study.md`,
- `artifacts/metrics/comparison-summary.json`,
- `artifacts/metrics/external-baseline-summary.json`,
- `artifacts/metrics/ablation-summary.json`,
- `artifacts/human_review/synthetic-review-summary.json`.

Within the manuscript directory, `python3` and the helper script `scripts/generate_tables.py` regenerate the checked-in table fragments from the JSON metric summaries. The `make` target then builds the PDF through `latexmk`. The current draft intentionally leaves bibliography enrichment as future work, because the repository does not yet carry a verified citation set.